



Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées de Khouribga

MINI PROJET JEE

Plateforme de Gestion et Analyse des Ventés

Réalisé par :

Ikram EL MENHI

Abderrahim AZEROUAL

Encadré par :

Pr. Noredline GHERABI

Année Universitaire : 2025–2026

Remerciements

Nous tenons à exprimer nos sincères remerciements et notre profonde gratitude à **Professeur Noredidine GHERABI**, notre encadrant, pour son accompagnement continu tout au long de ce travail. Sa disponibilité, ses conseils judicieux et son expertise ont été d'une valeur inestimable pour mener à bien ce projet. Nous lui sommes particulièrement reconnaissants pour sa patience, son écoute attentive et ses orientations méthodologiques qui nous ont permis de surmonter les difficultés rencontrées et d'approfondir nos réflexions.

Nos remerciements vont également à l'ensemble du corps professoral de l'**École Nationale des Sciences Appliquées de Khouribga** pour la qualité de la formation dispensée durant notre cursus. Les connaissances théoriques et pratiques acquises grâce à leur enseignement ont constitué une base solide pour la réalisation de ce projet. Nous apprécions particulièrement leur dévouement, leur rigueur pédagogique et leur engagement constant à transmettre leur savoir et à développer nos compétences.

Nous remercions aussi l'administration de l'ENSA Khouribga pour avoir mis à notre disposition les ressources et les infrastructures nécessaires à la réalisation de ce travail.

Enfin, nous exprimons notre reconnaissance à tous ceux qui, de près ou de loin, ont contribué à l'aboutissement de ce projet par leur soutien, leurs encouragements et leur bienveillance.

Résumé

Ce projet présente le développement d'une application web complète de gestion des ventes avec une architecture moderne comprenant un backend Java EE, un frontend React/TypeScript, et un système ETL Python pour l'analyse des données. L'application offre des interfaces spécialisées pour cinq types d'utilisateurs : administrateur, client, vendeur, analyste et investisseur.

Table des matières

Remerciements	1
Résumé	2
1 Introduction	7
1.1 Contexte du projet	7
1.2 Objectifs	7
1.3 Organisation du rapport	7
2 Architecture et Technologies	8
2.1 Architecture Générale	8
2.2 Architecture en Couches	8
2.2.1 Couches Backend (Java EE/Spring Boot)	8
2.2.2 Couches Frontend (React/TypeScript)	10
2.2.3 Communication entre les Couches	12
2.2.4 Avantages de cette Architecture	13
2.2.5 Structure du projet	13
2.2.6 Backend - Java EE	13
2.2.7 Frontend - React/TypeScript	13
2.2.8 ETL - Python	13
3 Méthodologie de Développement	14
3.1 Approche Agile Minimale	14
3.1.1 Principes directeurs	14
3.2 Organisation du travail	14
3.2.1 Phases du projet	14
3.3 Pratiques de développement	15
3.3.1 Gestion de version	15
3.3.2 Standards de code	15
3.3.3 Tests	16
3.4 Outils de collaboration	16
3.4.1 Outils de développement	16
3.4.2 Communication	16
3.5 Gestion des risques	16
3.5.1 Risques identifiés et mitigation	16
3.6 Livrables du projet	17
3.6.1 Livrables techniques	17
3.6.2 Livrables documentaires	17
4 Conception de l'Application	18

4.1	Modèle de données	18
4.2	Diagramme de classes	18
4.2.1	Description des entités principales	19
4.3	Architecture des APIs	20
5	Authentification et Page d'Accueil	21
5.1	Page d'Accueil (Landing Page)	21
5.1.1	Présentation	21
5.1.2	Caractéristiques	21
5.1.3	Capture d'écran	21
5.1.4	Implémentation	21
5.2	Page de Connexion (Login)	21
5.2.1	Présentation	21
5.2.2	Fonctionnalités	22
5.2.3	Capture d'écran	22
5.2.4	Implémentation	23
5.3	Page d'Inscription (Signup)	23
5.3.1	Présentation	23
5.3.2	Fonctionnalités	23
5.3.3	Capture d'écran	24
5.3.4	Implémentation	24
5.3.5	Sécurité	24
5.4	Flux d'Authentification Global	25
5.4.1	Diagramme de flux	25
5.4.2	Protection des Routes	25
6	Interface Administrateur	26
6.1	Présentation	26
6.2	Fonctionnalités	26
6.3	Captures d'écran	26
6.4	Implémentation	26
7	Interface Client	29
7.1	Présentation	29
7.2	Fonctionnalités	29
7.3	Captures d'écran	29
7.4	Implémentation	29
8	Interface Vendeur	31
8.1	Présentation	31
8.2	Fonctionnalités	31
8.3	Captures d'écran	31
8.4	Implémentation	31
9	Interface Analyste	33
9.1	Présentation	33
9.2	Fonctionnalités	33
9.3	Captures d'écran	33
9.4	Implémentation	33

10 Interface Investisseur	35
10.1 Présentation	35
10.2 Fonctionnalités	35
10.3 Captures d'écran	35
10.4 Implémentation	35
11 Implémentation Technique	38
11.1 Authentification et Sécurité	38
11.1.1 JWT Token	38
11.1.2 Configuration Spring Security	38
11.2 Gestion d'État avec React Context	38
12 Conclusion	40
12.1 Bilan du projet	40
12.2 Objectifs atteints	40
12.3 Difficultés rencontrées	40
12.4 Perspectives d'amélioration	40
12.5 Compétences acquises	41
Bibliographie	42
A Configuration du projet	43
A.1 Fichier pom.xml (Backend)	43
A.2 Fichier package.json (Frontend)	44
B Guide d'installation	45
B.1 Prérequis	45
B.2 Installation et lancement	45
B.2.1 Backend	45
B.2.2 Frontend	45
B.2.3 ETL Python	45

Table des figures

4.1	Diagramme de classes du système	18
5.1	Page d'accueil de l'application	22
5.2	Page de connexion	22
5.3	Page d'inscription	24
6.1	Tableau de bord administrateur	27
6.2	Gestion des utilisateurs	27
6.3	Gestion des produits	28
7.1	Catalogue de produits pour les clients	30
7.2	Historique des commandes	30
8.1	Tableau de bord vendeur	32
9.1	Tableau de bord analytique	34
9.2	Graphiques d'analyse	34
10.1	Tableau de bord investisseur	36
10.2	Opportunités d'investissement	36
10.3	Portefeuille d'investissements	37

Chapitre 1

Introduction

1.1 Contexte du projet

Dans le contexte actuel de digitalisation des entreprises, la gestion efficace des ventes et l'analyse des données sont devenues cruciales pour la compétitivité. Ce projet vise à développer une solution complète permettant la gestion multi-utilisateurs des ventes avec des fonctionnalités avancées d'analyse et de reporting.

1.2 Objectifs

Les principaux objectifs de ce projet sont :

- Développer une application web moderne et responsive
- Implémenter une gestion sécurisée des utilisateurs avec plusieurs rôles
- Créer des interfaces dédiées pour chaque type d'utilisateur
- Intégrer un système d'analyse de données (ETL)
- Assurer la sécurité et la performance de l'application

1.3 Organisation du rapport

Ce rapport est organisé en plusieurs chapitres :

- Chapitre 2 : Architecture et technologies utilisées
- Chapitre 3 : Méthodologie de développement
- Chapitre 4 : Conception de l'application
- Chapitres 5-9 : Présentation des interfaces utilisateurs
- Chapitre 10 : Implémentation technique
- Chapitre 11 : Conclusion

Chapitre 2

Architecture et Technologies

2.1 Architecture Générale

L'application suit une architecture trois tiers moderne :

- **Frontend** : React avec TypeScript et Vite
- **Backend** : Java EE avec Spring Boot
- **Base de données** : Système de gestion de base de données relationnelle
- **ETL** : Pipeline Python pour l'extraction, transformation et chargement des données

2.2 Architecture en Couches

L'application est structurée selon le modèle en couches qui assure une séparation claire des responsabilités, facilitant ainsi la maintenance et l'évolutivité du système.

2.2.1 Couches Backend (Java EE/Spring Boot)

Le backend est organisé en quatre couches principales :

Couche Contrôleur (Controller Layer)

Cette couche représente le point d'entrée des requêtes HTTP et gère les interactions avec les clients.

Responsabilités :

- Réception et validation des requêtes HTTP (GET, POST, PUT, DELETE)
- Mapping des endpoints REST avec les méthodes appropriées
- Gestion des paramètres de requête et du corps de la requête
- Transformation des réponses en format JSON
- Gestion des erreurs HTTP et codes de statut
- Application des annotations de sécurité (@PreAuthorize)

Exemples de contrôleurs :

- **AuthController** : Authentification et inscription
- **ProductController** : Gestion des produits

- `SaleController` : Opérations sur les ventes
- `DashboardController` : Données du tableau de bord
- `InvestmentController` : Gestion des investissements

Couche Service (Service Layer)

Contient la logique métier de l'application et orchestre les opérations complexes.

Responsabilités :

- Implémentation de la logique métier
- Orchestration des opérations entre plusieurs entités
- Gestion des transactions
- Validation des règles métier
- Transformation entre DTO et entités
- Gestion des exceptions métier

Exemples de services :

- `UserService` : Gestion des utilisateurs et rôles
- `ProductService` : Logique métier des produits
- `SaleService` : Calculs et statistiques de ventes
- `DashboardService` : Agrégation de données analytiques
- `InvestmentService` : Calculs financiers et ROI

Couche Repository (Data Access Layer)

Gère l'accès et la persistance des données.

Responsabilités :

- Opérations CRUD sur les entités
- Requêtes personnalisées (JPQL, SQL natif)
- Gestion des transactions base de données
- Abstraction de la base de données
- Pagination et tri des résultats

Exemples de repositories :

- `UserRepository` : Accès aux données utilisateurs
- `ProductRepository` : Requêtes sur les produits
- `SaleRepository` : Historique des ventes
- `ClientRepository` : Données clients
- `InvestmentRepository` : Opportunités d'investissement

Couche Modèle (Model Layer)

Définit les entités du domaine et les objets de transfert de données.

Composants :

- **Entités JPA** : Classes annotées représentant les tables
 - User, Product, Sale, Client, Investment
 - Annotations : @Entity, @Table, @Id, @Column, @ManyToOne, @OneToMany
- **DTO (Data Transfer Objects)** : Objets pour le transfert de données
 - UserDTO, SaleDTO, LoginRequest, JwtResponse
 - Séparation entre représentation interne et externe

Couche Sécurité (Security Layer)

Couche transversale gérant l'authentification et l'autorisation.

Composants :

- **JwtTokenProvider** : Génération et validation des tokens JWT
- **JwtAuthenticationFilter** : Filtre d'authentification des requêtes
- **SecurityConfig** : Configuration Spring Security
- **UserPrincipal** : Implémentation de UserDetails

2.2.2 Couches Frontend (React/TypeScript)

Le frontend est structuré selon une architecture modulaire par domaines fonctionnels.

Couche API (API Layer)

Interface de communication avec le backend.

Responsabilités :

- Configuration Axios avec intercepteurs
- Gestion automatique du token JWT dans les headers
- Définition des endpoints API
- Gestion centralisée des erreurs HTTP
- Transformation des réponses

Modules API :

- **axiosConfig.ts** : Configuration de base Axios
- **authApi.ts** : Authentification
- **productApi.ts** : Gestion des produits
- **saleApi.ts** : Opérations de vente
- **dashboardApi.ts** : Données analytiques
- **investmentApi.ts** : Services d'investissement

Couche Services/Contexte (State Management Layer)

Gestion de l'état global de l'application.

Composants :

— Contextes React :

- AuthContext : État d'authentification global
- Gestion de l'utilisateur connecté
- Persistance de session

— Hooks personnalisés :

- useAuth : Accès au contexte d'authentification
- useLogin : Logique de connexion
- useLogout : Logique de déconnexion

Couche Composants (Component Layer)

Composants React réutilisables et pages de l'application.

Organisation :

— Composants communs (components/) :

- Header, Footer : Layout de base
- Login, Signup : Authentification
- IntelligentSearchBar : Recherche globale
- Composants UI réutilisables (boutons, modales, etc.)

— Composants spécialisés :

- charts/ : Graphiques et visualisations
- forms/ : Formulaire de saisie
- tables/ : Tableaux de données

— Pages (pages/) :

- AdminDashboard : Interface administrateur
- ClientDashboard : Interface client
- VendorDashboard : Interface vendeur
- AnalysteDashboard : Interface analyste
- InvestorDashboard : Interface investisseur

Couche Routing (Navigation Layer)

Gestion de la navigation et des routes.

Fonctionnalités :

- Définition des routes de l'application
- Protection des routes (PrivateRoute)
- Redirection basée sur les rôles
- Navigation conditionnelle
- Gestion du 404

Couche Utilitaires (Utils Layer)

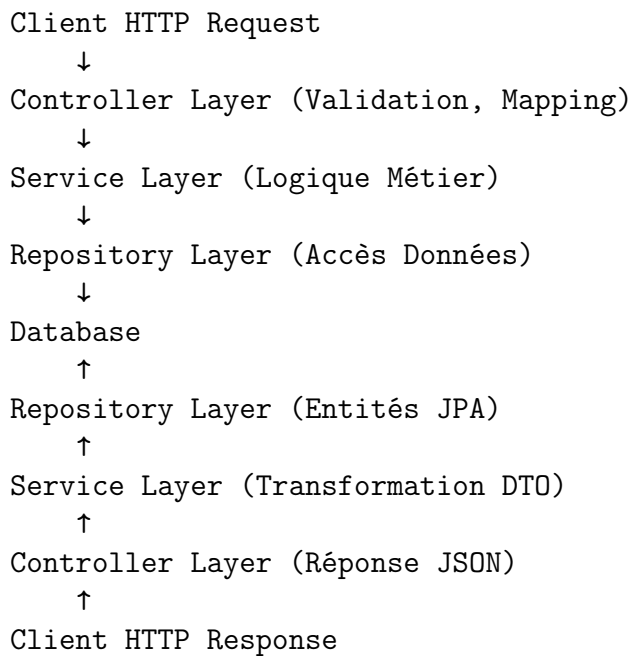
Fonctions et helpers réutilisables.

Contenu :

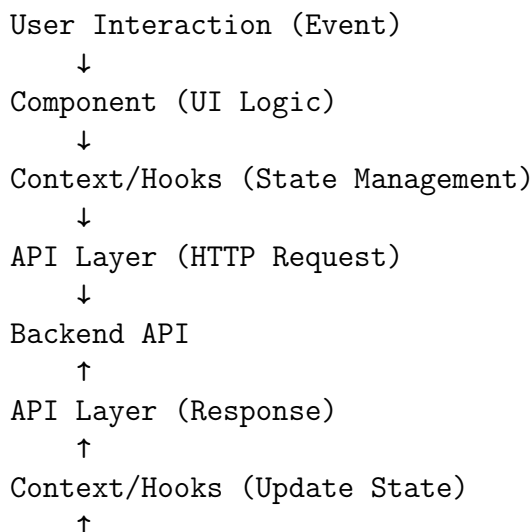
- Formateurs de dates et nombres
- Validateurs de formulaires
- Fonctions de calcul
- Constantes et configurations
- Types TypeScript personnalisés

2.2.3 Communication entre les Couches

Flux Backend



Flux Frontend



Component (Re-render)

↑

UI Update

2.2.4 Avantages de cette Architecture

- **Séparation des responsabilités** : Chaque couche a un rôle bien défini
- **Testabilité** : Possibilité de tester chaque couche indépendamment
- **Maintenabilité** : Modifications localisées sans impact global
- **Réutilisabilité** : Composants et services réutilisables
- **Scalabilité** : Facilité d'ajout de nouvelles fonctionnalités
- **Évolutivité** : Possibilité de remplacer une couche sans affecter les autres

2.2.5 Structure du projet

2.2.6 Backend - Java EE

- Spring Boot pour le framework
- Maven pour la gestion des dépendances
- JPA/Hibernate pour la persistance
- Spring Security pour l'authentification

2.2.7 Frontend - React/TypeScript

- React 18 avec TypeScript
- Vite comme build tool
- Tailwind CSS pour le design
- Axios pour les appels API
- React Router pour la navigation

2.2.8 ETL - Python

- Pandas pour la manipulation de données
- Scripts de nettoyage et transformation
- Pipeline automatisé

Chapitre 3

Méthodologie de Développement

3.1 Approche Agile Minimale

Pour ce projet, nous avons adopté une méthodologie agile minimale inspirée des principes Lean et Scrum, adaptée aux contraintes d'un projet académique. Cette approche nous a permis de maintenir une flexibilité dans le développement tout en assurant une progression structurée.

3.1.1 Principes directeurs

- **Développement itératif** : Le projet a été divisé en itérations courtes de 2 semaines
- **Livraison continue** : Chaque itération produit des fonctionnalités opérationnelles
- **Feedback régulier** : Points de contrôle hebdomadaires avec l'encadrant
- **Simplicité** : Privilégier les solutions simples et efficaces
- **Amélioration continue** : Rétrospectives après chaque itération

3.2 Organisation du travail

3.2.1 Phases du projet

Le projet a été structuré en quatre phases principales :

Phase 1 : Initialisation (Semaine 1-2)

- Analyse des besoins et spécifications
- Choix des technologies
- Mise en place de l'environnement de développement
- Création de la structure du projet
- Initialisation du dépôt Git

Phase 2 : Développement Backend (Semaine 3-6)

- Conception et création du modèle de données
- Implémentation des entités et repositories

- Développement des services métier
- Création des APIs REST
- Configuration de Spring Security et JWT
- Tests unitaires des services

Phase 3 : Développement Frontend (Semaine 7-10)

- Mise en place de la structure React/TypeScript
- Développement des composants réutilisables
- Intégration avec les APIs backend
- Implémentation des interfaces utilisateurs
- Gestion de l'authentification côté client
- Tests d'intégration

Phase 4 : ETL et Finalisation (Semaine 11-12)

- Développement du pipeline ETL Python
- Tests de bout en bout
- Optimisation des performances
- Documentation technique
- Préparation de la démonstration

3.3 Pratiques de développement

3.3.1 Gestion de version

- Utilisation de Git pour le contrôle de version
- Stratégie de branches : `main`, `develop`, `feature/*`
- Commits atomiques avec messages descriptifs
- Pull requests pour la revue de code
- GitHub pour l'hébergement du dépôt

3.3.2 Standards de code

- **Backend Java** : Respect des conventions Java et Spring Boot
- **Frontend TypeScript** : Utilisation d'ESLint et Prettier
- **Python** : Conformité aux standards PEP 8
- Documentation inline du code complexe
- Nommage cohérent et descriptif

3.3.3 Tests

- Tests unitaires pour les services backend (JUnit)
- Tests d'intégration des APIs
- Tests manuels des interfaces utilisateurs
- Validation des flux de travail complets

3.4 Outils de collaboration

3.4.1 Outils de développement

- **IDE** : IntelliJ IDEA (Backend), VS Code (Frontend/Python)
- **API Testing** : Postman pour les tests d'APIs
- **Base de données** : PostgreSQL/MySQL avec pgAdmin/Workbench
- **Versioning** : Git et GitHub

3.4.2 Communication

- Réunions hebdomadaires avec l'encadrant
- Sessions de pair programming pour les fonctionnalités complexes
- Documentation partagée sur GitHub (README, Wiki)
- Suivi des tâches via GitHub Issues

3.5 Gestion des risques

3.5.1 Risques identifiés et mitigation

Risque	Impact	Mitigation
Complexité technique	Retards de développement	Formation continue, utilisation de documentation
Incompatibilité des technologies	Problèmes d'intégration	Tests d'intégration précoces
Dérive des objectifs	Livraison incomplète	Priorisation stricte des fonctionnalités
Problèmes de performance	Expérience utilisateur dégradée	Optimisation et tests de charge

TABLE 3.1 – Matrice des risques et stratégies de mitigation

3.6 Livrables du projet

3.6.1 Livrables techniques

- Code source complet (Backend, Frontend, ETL)
- Scripts de base de données
- Configuration Docker pour le déploiement
- Documentation technique (README, commentaires)
- Tests unitaires et d'intégration

3.6.2 Livrables documentaires

- Ce rapport de projet
- Diagrammes UML (classes, séquence, cas d'utilisation)
- Documentation des APIs (format OpenAPI/Swagger)
- Guide d'installation et de déploiement
- Présentation de soutenance

Conception de l'Application

L'application gère plusieurs entités principales :

- Utilisateurs (avec différents rôles)
- Produits
- Clients
- Ventes
- Investissements
- Évaluations

Le diagramme de classes ci-dessous illustre les principales entités du système et leurs relations :



4.2.1 Description des entités principales

Utilisateur

La classe **User** représente les utilisateurs du système avec les attributs suivants :

- **id** : Identifiant unique
- **username** : Nom d'utilisateur
- **email** : Adresse email
- **password** : Mot de passe chiffré
- **role** : Rôle de l'utilisateur (ADMIN, CLIENT, VENDEUR, ANALYSTE, INVESTISSEUR)

Produit

La classe **Product** représente les produits disponibles à la vente :

- **id** : Identifiant unique
- **name** : Nom du produit
- **description** : Description détaillée
- **price** : Prix unitaire
- **stock** : Quantité en stock
- **category** : Catégorie du produit

Vente

La classe **Sale** enregistre les transactions de vente :

- **id** : Identifiant unique
- **date** : Date de la vente
- **quantity** : Quantité vendue
- **totalAmount** : Montant total
- **vendor** : Référence vers le vendeur
- **client** : Référence vers le client
- **product** : Référence vers le produit

Client

La classe **Client** contient les informations des clients :

- **id** : Identifiant unique
- **name** : Nom complet
- **email** : Adresse email
- **phone** : Numéro de téléphone
- **address** : Adresse postale

Investissement

La classe **Investment** représente les opportunités d'investissement :

- **id** : Identifiant unique
- **title** : Titre de l'opportunité
- **description** : Description détaillée
- **requiredAmount** : Montant requis
- **currentAmount** : Montant actuel collecté
- **expectedReturn** : Retour sur investissement attendu
- **status** : Statut (OPEN, CLOSED, FUNDED)

4.3 Architecture des APIs

Les APIs REST sont organisées par domaine fonctionnel avec les endpoints suivants :

- `/api/auth` : Authentification
- `/api/products` : Gestion des produits
- `/api/sales` : Gestion des ventes
- `/api/clients` : Gestion des clients
- `/api/dashboard` : Données du tableau de bord
- `/api/investments` : Gestion des investissements

Chapitre 5

Authentification et Page d'Accueil

5.1 Page d'Accueil (Landing Page)

5.1.1 Présentation

La page d'accueil constitue le point d'entrée principal de l'application. Elle présente une vue d'ensemble de la plateforme et offre aux visiteurs un accès direct aux fonctionnalités d'authentification.

5.1.2 Caractéristiques

- Design moderne et responsive adapté à tous les appareils
- Présentation claire des fonctionnalités de la plateforme
- Navigation intuitive vers les pages d'inscription et de connexion
- Mise en avant des différents rôles utilisateurs disponibles
- Section de présentation des avantages de la plateforme
- Footer avec informations de contact et liens utiles

5.1.3 Capture d'écran

5.1.4 Implémentation

La page d'accueil est développée avec React et TypeScript, utilisant Tailwind CSS pour le design responsive. Elle intègre :

- Composants Header et Footer réutilisables
- Sections présentant les fonctionnalités par rôle
- Animations et transitions pour améliorer l'expérience utilisateur
- Intégration avec React Router pour la navigation

5.2 Page de Connexion (Login)

5.2.1 Présentation

La page de connexion permet aux utilisateurs existants d'accéder à leur compte en s'authentifiant avec leurs identifiants.

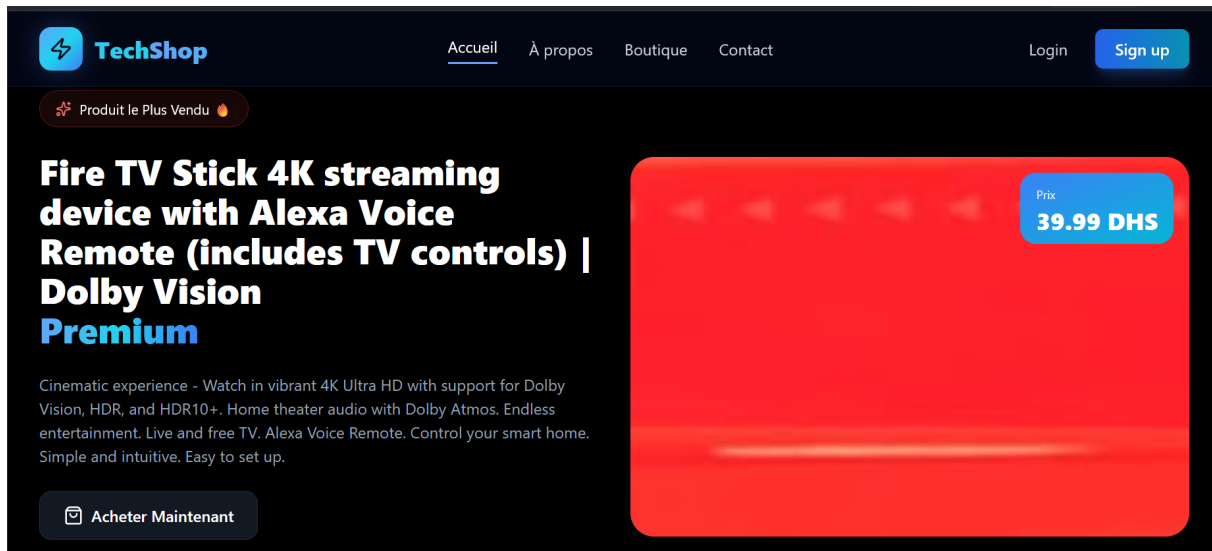


FIGURE 5.1 – Page d'accueil de l'application

5.2.2 Fonctionnalités

- Formulaire de connexion sécurisé
- Champs pour nom d'utilisateur/email et mot de passe
- Validation côté client des champs
- Gestion des erreurs d'authentification
- Messages d'erreur contextuels et clairs
- Redirection automatique selon le rôle après connexion
- Option "Se souvenir de moi" (optionnel)
- Lien vers la page d'inscription

5.2.3 Capture d'écran

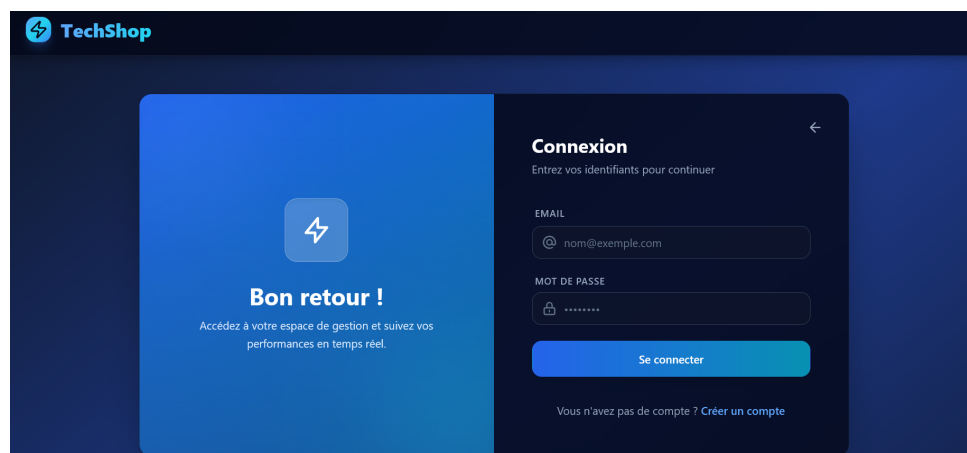


FIGURE 5.2 – Page de connexion

5.2.4 Implémentation

Le composant Login est implémenté dans `Login.tsx` et utilise :

- **Hook personnalisé useLogin** : Gère la logique d'authentification
- **Validation de formulaire** : Vérification des champs avant soumission
- **Gestion d'état** : Utilisation de `useState` pour les champs et messages d'erreur
- **API d'authentification** : Appel à l'endpoint `/api/auth/login`
- **Stockage du token JWT** : Sauvegarde dans `localStorage` via `AuthContext`
- **Redirection conditionnelle** : Navigation vers le dashboard approprié selon le rôle

Le processus d'authentification :

1. L'utilisateur saisit ses identifiants
2. Validation côté client des données
3. Envoi de la requête POST au backend
4. Backend vérifie les credentials et génère un JWT
5. Frontend stocke le token et met à jour le contexte d'authentification
6. Redirection vers l'interface correspondant au rôle de l'utilisateur

Référence : `gestionventesfrontend/src/components/Login.tsx`

5.3 Page d'Inscription (Signup)

5.3.1 Présentation

La page d'inscription permet aux nouveaux utilisateurs de créer un compte sur la plateforme.

5.3.2 Fonctionnalités

- Formulaire d'inscription complet
- Champs : nom d'utilisateur, email, mot de passe, confirmation de mot de passe
- Validation en temps réel des champs
- Vérification de la force du mot de passe
- Confirmation de mot de passe avec vérification de correspondance
- Validation de l'unicité de l'email et du nom d'utilisateur
- Messages d'erreur détaillés et contextuels
- Indication visuelle des champs valides/invalides
- Lien vers la page de connexion pour les utilisateurs existants

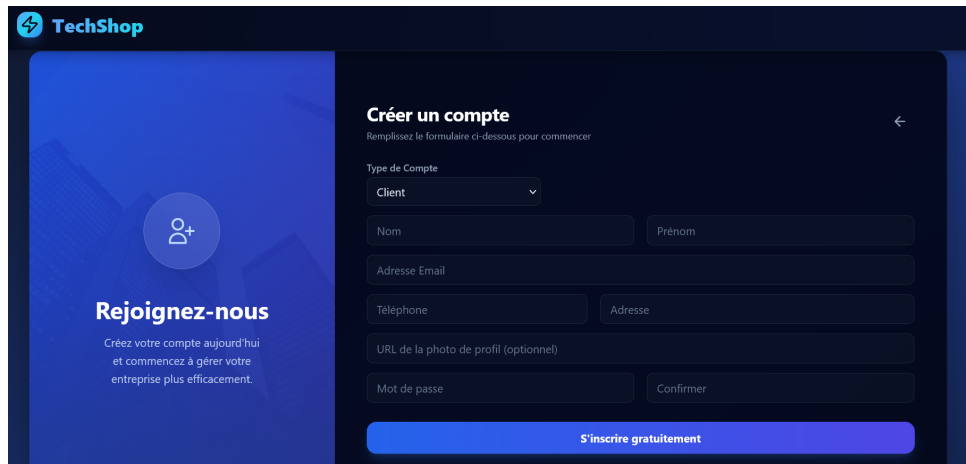


FIGURE 5.3 – Page d’inscription

5.3.3 Capture d’écran

5.3.4 Implémentation

Le composant Signup est implémenté dans `Signup.tsx` et intègre :

- **Hook personnalisé useSignup** : Gère la logique d’inscription
- **Validation avancée** :
 - Vérification du format email (regex)
 - Validation de la force du mot de passe (longueur minimale, caractères spéciaux)
 - Confirmation du mot de passe
 - Vérification de l’unicité du nom d’utilisateur
- **Feedback utilisateur** : Indicateurs visuels de validation
- **API d’inscription** : Appel à l’endpoint `/api/auth/signup`
- **Gestion des erreurs** : Affichage des erreurs backend (email déjà utilisé, etc.)
- **Redirection automatique** : Connexion et redirection après inscription réussie

Le processus d’inscription :

1. L’utilisateur remplit le formulaire d’inscription
2. Validation côté client de toutes les données
3. Envoi de la requête POST au backend
4. Backend vérifie l’unicité de l’email/username
5. Création du compte avec mot de passe chiffré (BCrypt)
6. Génération automatique d’un token JWT
7. Frontend stocke le token et authentifie l’utilisateur
8. Redirection vers l’interface appropriée

5.3.5 Sécurité

Les mesures de sécurité implémentées incluent :

- Chiffrement du mot de passe côté backend (BCrypt avec salt)

- Validation stricte des données côté serveur
- Protection contre les injections SQL (JPA/Hibernate)
- Vérification de l'unicité des identifiants
- Tokens JWT sécurisés pour l'authentification
- Expiration automatique des sessions

Référence : *gestionventesfrontend/src/components/Signup.tsx*

5.4 Flux d'Authentification Global

5.4.1 Diagramme de flux

Le processus d'authentification suit le schéma suivant :

Landing Page



Connexion (Login)

Validation des credentials

Génération JWT

Redirection selon rôle

Inscription (Signup)

Validation des données

Création du compte

Authentification automatique

Redirection selon rôle

Redirection par rôle :

ADMIN → AdminDashboard

CLIENT → ClientDashboard

VENDEUR → VendorDashboard

ANALYSTE → AnalysteDashboard

INVESTISSEUR → InvestorDashboard

5.4.2 Protection des Routes

L'application utilise un système de routes protégées qui :

- Vérifie la présence d'un token JWT valide
- Contrôle les autorisations basées sur les rôles
- Redirige vers la page de connexion si non authentifié
- Empêche l'accès non autorisé aux interfaces

Chapitre 6

Interface Administrateur

6.1 Présentation

L'interface administrateur offre un contrôle complet sur l'application. L'administrateur peut gérer tous les utilisateurs, configurer les paramètres système, et accéder à tous les modules.

6.2 Fonctionnalités

- Gestion des utilisateurs (CRUD)
- Attribution des rôles
- Configuration du système
- Visualisation des statistiques globales
- Gestion des catégories
- Supervision des ventes

6.3 Captures d'écran

6.4 Implémentation

Le service de gestion des utilisateurs est implémenté dans la classe `UserService.java` qui se trouve dans le package `com.gestionventes.service`. Cette classe gère les opérations CRUD sur les utilisateurs avec les fonctionnalités suivantes :

- `createUser()` : Création d'un nouvel utilisateur avec chiffrement du mot de passe
- `getAllUsers()` : Récupération de la liste complète des utilisateurs
- `updateUser()` : Modification des informations d'un utilisateur
- `deleteUser()` : Suppression d'un utilisateur par son identifiant

Référence : `gestionventesbackend/src/main/java/com/gestionventes/service/UserService.java`



FIGURE 6.1 – Tableau de bord administrateur

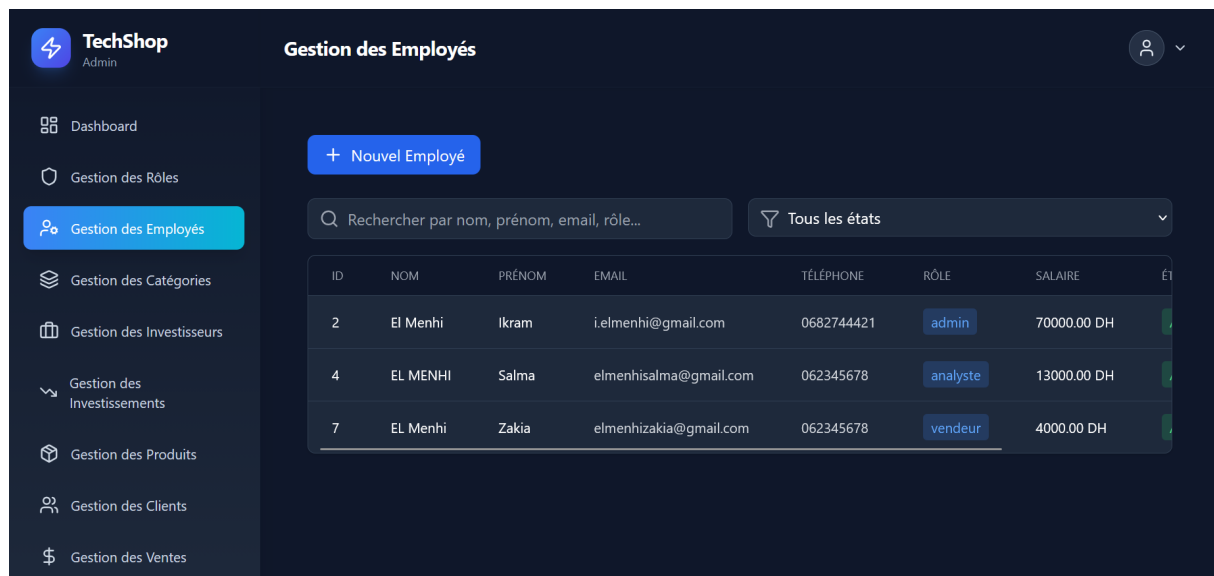
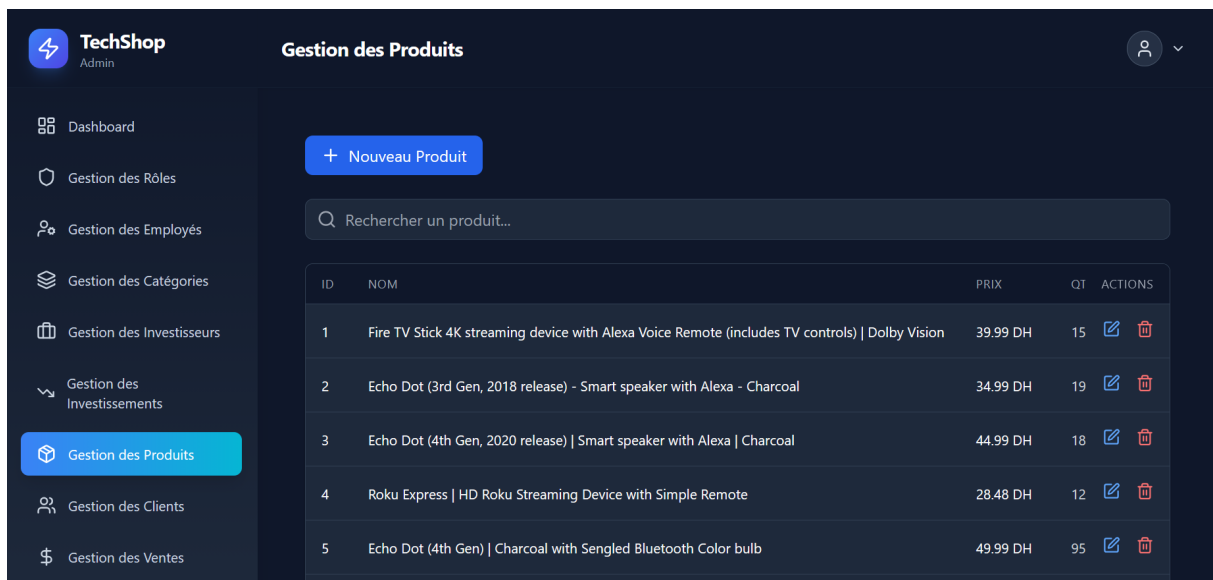


FIGURE 6.2 – Gestion des utilisateurs



TechShop
Admin

Gestion des Produits

+ Nouveau Produit

Rechercher un produit...

ID	NOM	PRIX	QT	ACTIONS
1	Fire TV Stick 4K streaming device with Alexa Voice Remote (includes TV controls) Dolby Vision	39.99 DH	15	 
2	Echo Dot (3rd Gen, 2018 release) - Smart speaker with Alexa - Charcoal	34.99 DH	19	 
3	Echo Dot (4th Gen, 2020 release) Smart speaker with Alexa Charcoal	44.99 DH	18	 
4	Roku Express HD Roku Streaming Device with Simple Remote	28.48 DH	12	 
5	Echo Dot (4th Gen) Charcoal with Sengled Bluetooth Color bulb	49.99 DH	95	 

FIGURE 6.3 – Gestion des produits

Chapitre 7

Interface Client

7.1 Présentation

L'interface client permet aux clients de consulter le catalogue de produits, passer des commandes et suivre leurs achats.

7.2 Fonctionnalités

- Consultation du catalogue de produits
- Recherche et filtrage des produits
- Visualisation des détails produits
- Passage de commandes
- Historique des achats
- Évaluation des produits

7.3 Captures d'écran

7.4 Implémentation

Le composant de recherche intelligente est développé en TypeScript avec React. Il se trouve dans `IntelligentSearchBar.tsx` et offre les fonctionnalités suivantes :

- **Recherche en temps réel** : Utilisation d'un debounce de 300ms pour optimiser les requêtes
- **Recherche multi-entités** : Capacité de rechercher dans les produits, clients et ventes
- **Indicateur de chargement** : Feedback visuel pendant la recherche
- **Affichage des résultats** : Liste dynamique des résultats avec composant dédié

Référence : `gestionventesfrontend/src/components/IntelligentSearchBar.tsx`

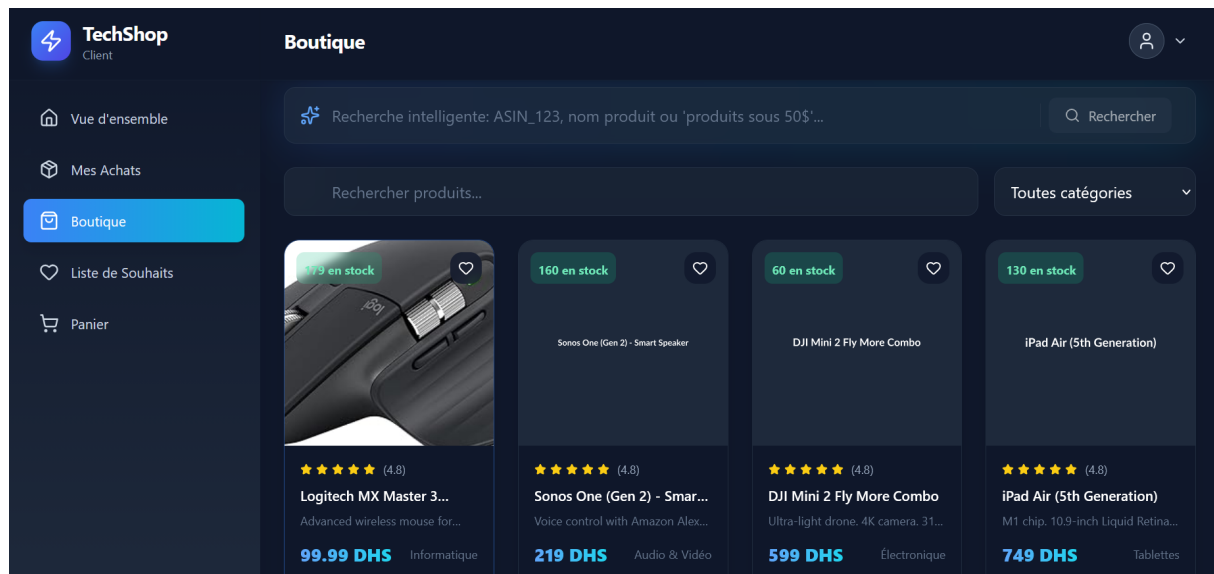


FIGURE 7.1 – Catalogue de produits pour les clients

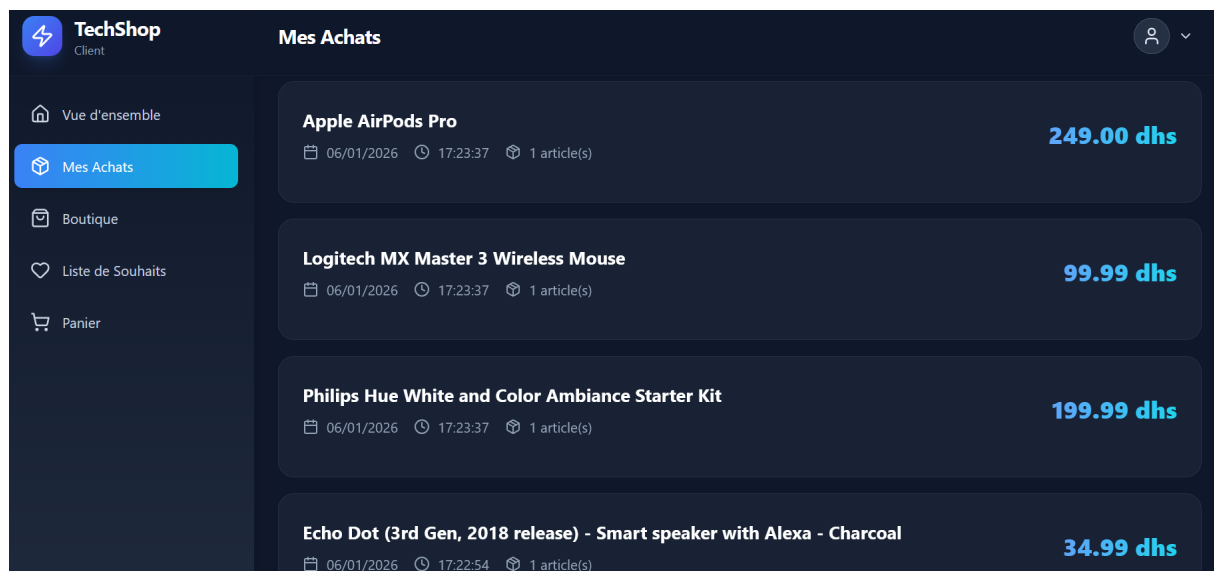


FIGURE 7.2 – Historique des commandes

Chapitre 8

Interface Vendeur

8.1 Présentation

L'interface vendeur est optimisée pour la gestion quotidienne des ventes et le suivi des clients.

8.2 Fonctionnalités

- Enregistrement des ventes
- Gestion du stock
- Consultation des produits
- Suivi des clients
- Statistiques de ventes personnelles
- Objectifs de vente

8.3 Captures d'écran

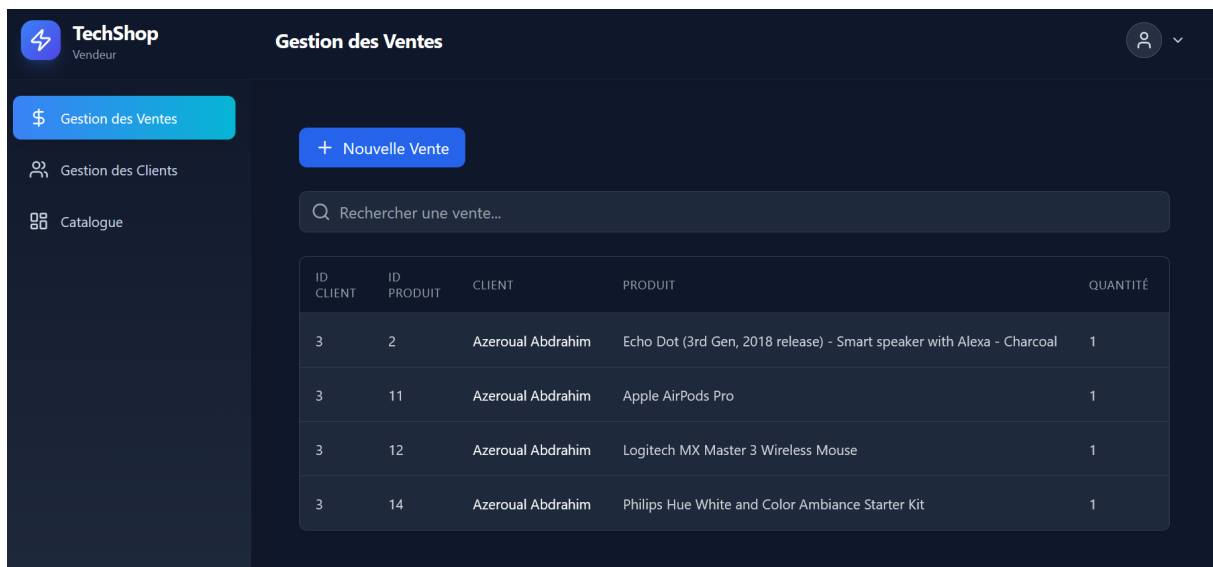
8.4 Implémentation

L'API de gestion des ventes est implémentée dans le contrôleur `SaleController.java`. Cette API REST expose les endpoints suivants :

- **POST** `/api/sales` : Création d'une nouvelle vente (accessible aux vendeurs et administrateurs)
- **GET** `/api/sales` : Récupération des ventes avec filtres optionnels (vendeur, période)
- **GET** `/api/sales/{id}` : Détails d'une vente spécifique
- **GET** `/api/sales/vendor/statistics` : Statistiques de performance par vendeur

Le contrôleur utilise Spring Security avec des annotations `@PreAuthorize` pour gérer les autorisations basées sur les rôles.

Référence : `gestionventesbackend/src/main/java/com/gestionventes/controller/SaleController.java`



The screenshot shows the 'Gestion des Ventes' (Sales Management) dashboard for a vendor named 'TechShop'. The interface is dark-themed. On the left, there is a sidebar with three menu items: 'Gestion des Ventes' (highlighted with a blue bar), 'Gestion des Clients', and 'Catalogue'. The main content area has a header with the title 'Gestion des Ventes' and a user profile icon. Below the header, there is a blue button labeled '+ Nouvelle Vente' (New Sale) and a search bar with the placeholder text 'Rechercher une vente...'. A table displays a list of sales records with the following columns: ID CLIENT, ID PRODUIT, CLIENT, PRODUIT, and QUANTITÉ. The table contains four rows of data, all for client 'Azeroual Abdrahim'.

ID CLIENT	ID PRODUIT	CLIENT	PRODUIT	QUANTITÉ
3	2	Azeroual Abdrahim	Echo Dot (3rd Gen, 2018 release) - Smart speaker with Alexa - Charcoal	1
3	11	Azeroual Abdrahim	Apple AirPods Pro	1
3	12	Azeroual Abdrahim	Logitech MX Master 3 Wireless Mouse	1
3	14	Azeroual Abdrahim	Philips Hue White and Color Ambiance Starter Kit	1

FIGURE 8.1 – Tableau de bord vendeur

Chapitre 9

Interface Analyste

9.1 Présentation

L'interface analyste offre des outils avancés de visualisation et d'analyse des données de ventes.

9.2 Fonctionnalités

- Tableaux de bord analytiques
- Graphiques interactifs
- Rapports personnalisés
- Analyse des tendances
- Prévisions de ventes
- Export de données (CSV, Excel)

9.3 Captures d'écran

9.4 Implémentation

Le pipeline ETL est implémenté dans la classe Python `ETLPipeline` qui orchestre les trois phases du processus :

- **Extract()** : Extraction des données depuis des fichiers CSV
- **Transform()** : Transformation et nettoyage incluant :
 - Suppression des doublons
 - Nettoyage des valeurs nulles
 - Conversion des types de données
 - Calcul de colonnes dérivées (revenus, périodes)
- **Load()** : Chargement des données transformées vers la base de données
- **Run()** : Exécution complète du pipeline avec gestion des logs

Le pipeline utilise la bibliothèque Pandas pour la manipulation efficace des données et inclut une gestion complète des erreurs avec logging.

Référence : `gestionventesdata/etl/etl_pipeline.py`



FIGURE 9.1 – Tableau de bord analytique

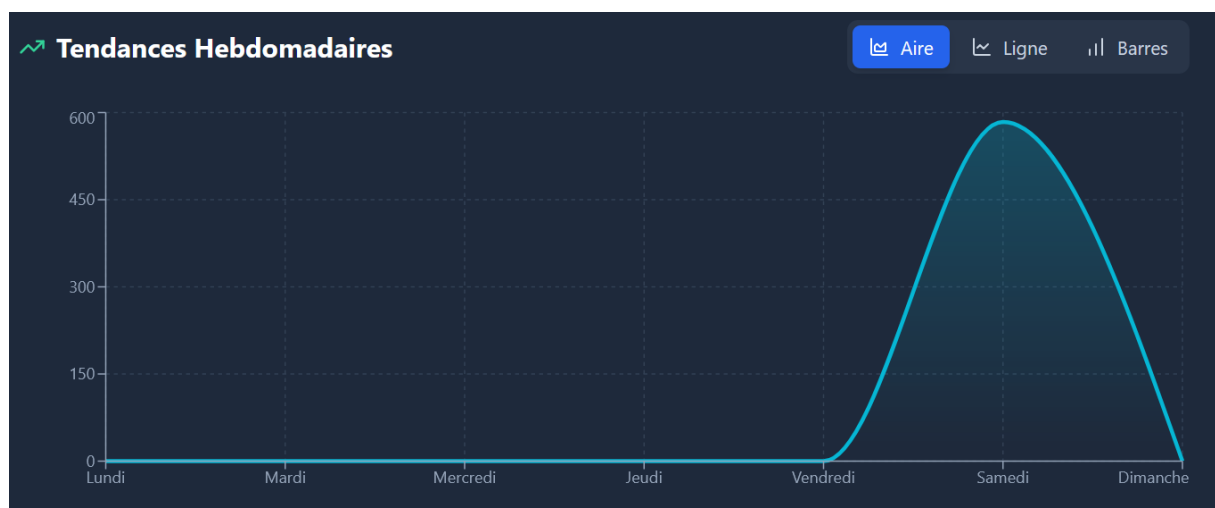


FIGURE 9.2 – Graphiques d'analyse

Chapitre 10

Interface Investisseur

10.1 Présentation

L'interface investisseur fournit des informations financières et des opportunités d'investissement.

10.2 Fonctionnalités

- Consultation des opportunités d'investissement
- Suivi du portefeuille
- Indicateurs financiers
- Rapports de performance
- Historique des investissements
- Prévisions de ROI

10.3 Captures d'écran

10.4 Implémentation

Le service d'investissement est développé en TypeScript et expose une API client pour les opérations d'investissement. Les interfaces définies incluent :

- **Investment** : Représente une opportunité d'investissement avec statut, montants et ROI
- **InvestmentContribution** : Données pour effectuer une contribution

Les méthodes disponibles sont :

- **getAllInvestments()** : Récupère toutes les opportunités
- **getOpenOpportunities()** : Filtre les opportunités ouvertes
- **makeInvestment()** : Effectue un investissement
- **getInvestorPortfolio()** : Portfolio d'un investisseur
- **getInvestmentStatistics()** : Statistiques d'investissement

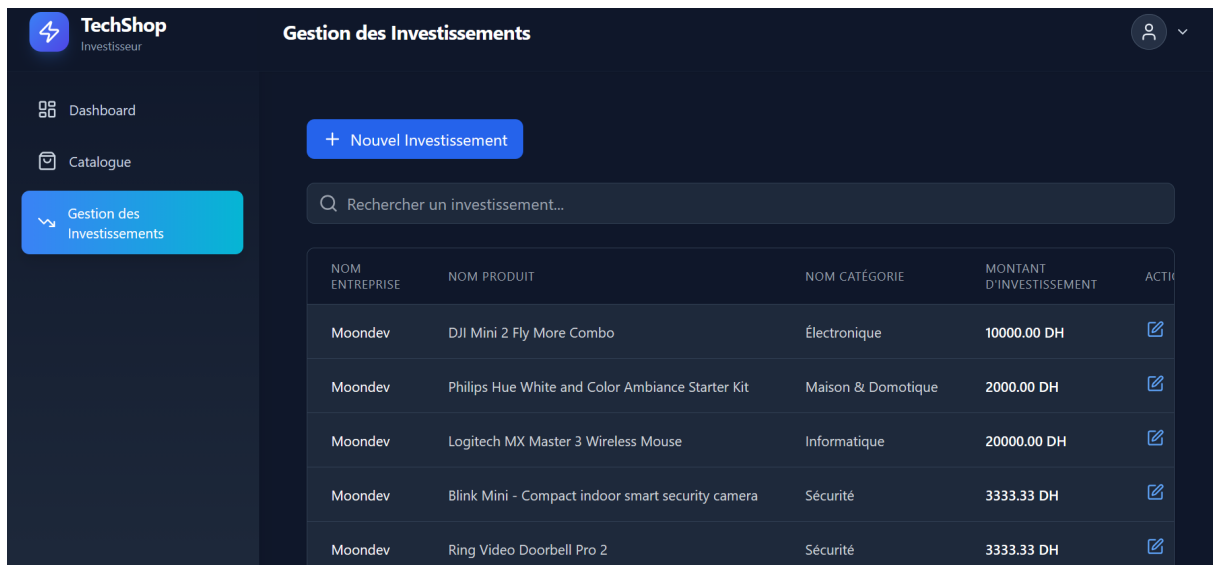
Référence : *gestionventesfrontend/src/api/investmentApi.ts*



FIGURE 10.1 – Tableau de bord investisseur

Rang	Produit	Catégorie	Prix	Ventes	Quantité	Revenus	Score
#1	Fire TV Stick 4K streaming device with Alexa Voice Remote (includes TV controls) Dolby Vision	N/A	39.99 DH	0	0	0 DH	Hot
#2	Echo Dot (3rd Gen, 2018 release) - Smart speaker with Alexa - Charcoal	N/A	34.99 DH	1	0	34,99 DH	Hot
#3	Echo Dot (4th Gen, 2020 release) Smart speaker with Alexa Charcoal	N/A	44.99 DH	0	0	0 DH	Hot
#4	Roku Express HD Roku Streaming Device with Simple Remote	N/A	28.48 DH	0	0	0 DH	Fort

FIGURE 10.2 – Opportunités d'investissement



NOM ENTREPRISE	NOM PRODUIT	NOM CATÉGORIE	MONTANT D'INVESTISSEMENT	ACTI
Moondév	DJI Mini 2 Fly More Combo	Électronique	10000.00 DH	
Moondév	Philips Hue White and Color Ambiance Starter Kit	Maison & Domotique	2000.00 DH	
Moondév	Logitech MX Master 3 Wireless Mouse	Informatique	20000.00 DH	
Moondév	Blink Mini - Compact indoor smart security camera	Sécurité	3333.33 DH	
Moondév	Ring Video Doorbell Pro 2	Sécurité	3333.33 DH	

FIGURE 10.3 – Portefeuille d'investissements

Chapitre 11

Implémentation Technique

11.1 Authentification et Sécurité

11.1.1 JWT Token

L'application utilise JWT (JSON Web Tokens) pour l'authentification. La classe `JwtTokenProvider` gère :

- **`generateToken()`** : Génération d'un token JWT avec expiration configurable
- **`getUserIdFromToken()`** : Extraction de l'ID utilisateur depuis le token
- **`validateToken()`** : Validation de l'intégrité et de l'expiration du token

Le token est signé avec l'algorithme HS512 et contient l'identifiant de l'utilisateur, la date d'émission et d'expiration.

Référence : `gestionventesbackend/src/main/java/com/gestionventes/security/JwtTokenProvider.java`

11.1.2 Configuration Spring Security

La configuration de sécurité est définie dans `SecurityConfig.java` et implémente :

- Désactivation de CSRF (application stateless)
- Gestion de session STATELESS
- Configuration des endpoints publics (`/api/auth/**`)
- Protection des endpoints par rôles :
 - `/api/admin/**` : ADMIN uniquement
 - `/api/vendor/**` : VENDEUR et ADMIN
 - `/api/analyst/**` : ANALYSTE et ADMIN
 - `/api/investor/**` : INVESTISSEUR et ADMIN
- Filtre JWT pour l'authentification des requêtes

Référence : `gestionventesbackend/src/main/java/com/gestionventes/security/SecurityConfig.java`

11.2 Gestion d'État avec React Context

L'authentification côté client est gérée via React Context API dans `AuthContext.tsx`. Ce contexte fournit :

- **AuthProvider** : Composant wrapper qui gère l'état global d'authentification
- **useAuth** : Hook personnalisé pour accéder au contexte d'authentification

Fonctionnalités implémentées :

- **Persistence de session** : Vérification automatique du token au chargement
- **login()** : Authentification et stockage du token JWT
- **logout()** : Déconnexion et nettoyage du localStorage
- **isAuthenticated** : État booléen de connexion
- **isLoading** : Indicateur de chargement pour l'initialisation

Le token JWT est stocké dans le localStorage et automatiquement inclus dans les en-têtes des requêtes API via Axios.

Référence : gestionventesfrontend/src/contexts/AuthContext.tsx

Chapitre 12

Conclusion

12.1 Bilan du projet

Ce projet a permis de développer une application web complète de gestion des ventes avec une architecture moderne et scalable. L'implémentation de cinq interfaces utilisateurs distinctes (administrateur, client, vendeur, analyste, investisseur) répond aux besoins spécifiques de chaque type d'utilisateur.

12.2 Objectifs atteints

- Développement d'une architecture trois tiers robuste
- Implémentation complète des fonctionnalités CRUD
- Sécurisation de l'application avec JWT
- Création d'interfaces utilisateur modernes et responsives
- Intégration d'un pipeline ETL pour l'analyse des données
- Tests unitaires et d'intégration
- Containerisation avec Docker

12.3 Difficultés rencontrées

- Gestion des permissions multi-rôles
- Optimisation des requêtes base de données
- Synchronisation des données en temps réel
- Configuration du pipeline ETL

12.4 Perspectives d'amélioration

Plusieurs améliorations peuvent être envisagées pour les versions futures :

- Implémentation de WebSockets pour les notifications en temps réel
- Ajout d'un système de messagerie interne
- Développement d'une application mobile

- Intégration avec des services de paiement en ligne
- Amélioration des algorithmes de prévision
- Mise en place de tests end-to-end automatisés
- Implémentation d'un système de cache distribué (Redis)
- Ajout de l'internationalisation (i18n)

12.5 Compétences acquises

Ce projet a permis d'acquérir et de renforcer les compétences suivantes :

- Développement full-stack (Java EE, React, TypeScript)
- Architecture microservices
- Gestion de base de données relationnelles
- Sécurité des applications web
- ETL et analyse de données avec Python
- DevOps et containerisation
- Méthodologie Agile

Bibliographie

- [1] Documentation officielle Spring Boot, <https://spring.io/projects/spring-boot>
- [2] Documentation officielle React, <https://react.dev/>
- [3] Documentation TypeScript, <https://www.typescriptlang.org/docs/>
- [4] Introduction to JSON Web Tokens, <https://jwt.io/introduction>
- [5] Pandas Documentation, <https://pandas.pydata.org/docs/>
- [6] Tailwind CSS Documentation, <https://tailwindcss.com/docs>

Chapitre A

Configuration du projet

A.1 Fichier pom.xml (Backend)

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0">
3   <modelVersion>4.0.0</modelVersion>
4
5   <groupId>com.gestionventes</groupId>
6   <artifactId>gestionventes-backend</artifactId>
7   <version>1.0.0</version>
8
9   <parent>
10     <groupId>org.springframework.boot</groupId>
11     <artifactId>spring-boot-starter-parent</artifactId>
12     <version>2.7.0</version>
13   </parent>
14
15   <dependencies>
16     <dependency>
17       <groupId>org.springframework.boot</groupId>
18       <artifactId>spring-boot-starter-web</artifactId>
19     </dependency>
20     <dependency>
21       <groupId>org.springframework.boot</groupId>
22       <artifactId>spring-boot-starter-data-jpa</artifactId>
23     </dependency>
24     <dependency>
25       <groupId>org.springframework.boot</groupId>
26       <artifactId>spring-boot-starter-security</artifactId>
27     </dependency>
28     <dependency>
29       <groupId>io.jsonwebtoken</groupId>
30       <artifactId>jjwt</artifactId>
31       <version>0.9.1</version>
32     </dependency>
33   </dependencies>
34 </project>
```

A.2 Fichier package.json (Frontend)

```
1 {
2   "name": "gestionventes-frontend",
3   "version": "1.0.0",
4   "type": "module",
5   "scripts": {
6     "dev": "vite",
7     "build": "tsc && vite build",
8     "preview": "vite preview"
9   },
10  "dependencies": {
11    "react": "^18.2.0",
12    "react-dom": "^18.2.0",
13    "react-router-dom": "^6.11.0",
14    "axios": "^1.4.0",
15    "tailwindcss": "^3.3.0"
16  },
17  "devDependencies": {
18    "@types/react": "^18.2.0",
19    "@types/react-dom": "^18.2.0",
20    "@vitejs/plugin-react": "^4.0.0",
21    "typescript": "^5.0.0",
22    "vite": "^4.3.0"
23  }
24 }
```

Chapitre B

Guide d'installation

B.1 Prérequis

- Java JDK 11 ou supérieur
- Node.js 18 ou supérieur
- Python 3.8 ou supérieur
- Maven 3.6+
- MySQL (ou autre SGBD)

B.2 Installation et lancement

B.2.1 Backend

```
cd gestionventesbackend
mvn clean install
mvn spring-boot:run
```

B.2.2 Frontend

```
cd gestionventesfrontend
npm install
npm run dev
```

B.2.3 ETL Python

```
cd gestionventesdata
pip install -r requirements.txt
python etl/etl_pipeline.py
```